

Guía de compilación y despliegue — UMG Personaliza

Entregables 1.1.2.3 (frontend) y 1.1.2.4 (backend): desde clonar en GitHub hasta tener el sistema funcionando.

Estos pasos se ejecutaron de verdad, clonando el repositorio desde cero en una máquina Windows 11 el 2026-08-31. Los problemas que aparecen en la sección de solución de problemas **ocurrieron realmente durante ese proceso**, no son hipótesis.

Requisitos

Herramienta	Versión	Para qué
Git	2.x	Clonar el repositorio
Docker Desktop	29.x o superior	Levantar los tres servicios
Node.js	20 o superior	Solo si trabajas sin Docker

Con Docker no hace falta instalar Node, MySQL ni Nginx en tu equipo: las imágenes los traen. El backend corre sobre **Node 20.20.2** y **npm 10.8.2** dentro del contenedor.

Comprueba que Docker esté **en ejecución**, no solo instalado:

```
docker info
```

Si responde `failed to connect to the docker API`, abre Docker Desktop y espera a que arranque.

Paso 1 — Clonar

```
git clone https://github.com/ediazc20-gif/UMG-Personaliza.git
cd UMG-Personaliza
```

El repositorio es privado, así que Git pedirá autenticación la primera vez. En Windows, Git Credential Manager abre una ventana del navegador para iniciar sesión en GitHub y guarda las credenciales para las siguientes veces.

Paso 2 — Crear el archivo de entorno

```
cp .env.example .env
```

Abre `.env` y **rellena estos dos como mínimo**, o el arranque falla:

```
MYSQL_ROOT_PASSWORD=<una contraseña fuerte>
JWT_SECRET=<una cadena larga y aleatoria>

# Debe coincidir con MYSQL_ROOT_PASSWORD
LOCAL_DB_PASSWORD=<la misma>
CENTRALP_DB_PASSWORD=<la misma>
```

`docker-compose.yml` está configurado para **fallar explícitamente** si faltan

`MYSQL_ROOT_PASSWORD` o `JWT_SECRET`, en vez de arrancar con valores inseguros por defecto.

Para generar un secreto aleatorio:

```
node -e "console.log(require('crypto').randomBytes(48).toString('hex'))"
```

`.env` está en `.gitignore`: nunca se sube al repositorio. Cada integrante crea el suyo.

Paso 3 — Compilar y levantar

```
docker compose up -d --build
```

La primera vez tarda varios minutos: el backend compila dependencias nativas

(`canvas` y TensorFlow para el reconocimiento facial) y necesita las bibliotecas

de sistema que instala su Dockerfile.

Esto construye y arranca tres servicios:

Servicio	Imagen	Puerto	Qué hace
<code>frontend</code>	<code>nginx:alpine</code>	8081 → 80	Sirve las vistas y hace de proxy inverso
<code>backend</code>	Node 20 (compilada del Dockerfile)	3000	API REST y WebSocket
<code>mysql</code>	<code>mysql:8.0</code>	3307 → 3306	Base de datos

El backend espera a que MySQL esté sano (`healthcheck`) antes de arrancar, así que

no hay que ordenar el arranque a mano.

Paso 4 — Comprobar que funcionó

```
docker compose ps
```

Los tres deben aparecer `Up`, y `mysql` además (`healthy`).

```
# El frontend responde
curl -o /dev/null -w "%{http_code}\n" http://localhost:8081/

# La API devuelve el catálogo
curl http://localhost:8081/api/tienda/productos

# El login funciona
curl -X POST http://localhost:8081/login \
  -H "Content-Type: application/json" \
  -d '{"usuario":"admin","contrasena":"admin123}"'
```

Y comprueba que la base se pobló sola:

```
docker compose exec -T mysql sh -c \
  'mysql -uroot -p"$MYSQL_ROOT_PASSWORD" umg_personaliza_db \
  -e "SELECT COUNT(*) AS usuarios FROM usuarios;"'
```

Debe devolver 5. Si devuelve 2, algún script de `database/` falló y cortó la cadena de inicialización — revisa `docker compose logs mysql | grep ERROR`.

Abre <http://localhost:8081> y entra con `admin` / `admin123`.

Compilación del frontend

El frontend es **JavaScript puro (ES6+)**, sin framework y sin paso de build. No hay

`npm run build`, ni bundler, ni transpilación.

`frontend/Dockerfile` hace solo dos cosas:

```
FROM nginx:alpine
COPY nginx.conf /etc/nginx/conf.d/default.conf
COPY src/ /usr/share/nginx/html/
```

Copia la configuración de Nginx y los archivos de `src/` tal cual.

Desarrollo en vivo

`docker-compose.yml` monta `./frontend/src` como volumen dentro del contenedor, así

que **al editar un archivo del frontend basta con recargar el navegador**. No hay que reconstruir nada.

La excepción es `nginx.conf`, que sí se copia en la imagen:

```
docker compose up -d --build frontend
```

Estructura

frontend/src/	
├ index.html	landing con acceso triple (contraseña, QR, facial)
├ admin/	panel de administración
├ comprador/	tienda, carrito, checkout, tracking, historial, perfil
├ repartidor/	vista de entrega
├ supervisor/	dashboard de ventas
├ css/	sistema de diseño (regla 60-30-10)
├ js/	cliente de API, auth, cámara, reconocimiento facial
├ lib/	GSAP, ScrollTrigger, Lenis
├ models/	modelos de detección y reconocimiento facial

Compilación del backend

backend/Dockerfile usa **construcción en dos etapas**:

1. **Etapla builder** — parte de `node:20-bullseye`, instala las bibliotecas de sistema que necesitan las dependencias nativas (`libcairo2-dev`, `libpango1.0-dev`, `libjpeg-dev`, `libgif-dev`, `librsvg2-dev`, más `python3`, `make` y `g++`) y ejecuta `npm install --omit=dev`.

1. **Etapla final** — parte de la misma imagen base limpia y copia solo los `node_modules` ya compilados y el código.

Así la imagen final no arrastra los compiladores ni las cabeceras de desarrollo.

Esas bibliotecas hacen falta porque `canvas` y `@tensorflow/tfjs` se compilan nativamente; sin ellas `npm install` falla con errores de `node-gyp`.

Reconstruir tras cambiar dependencias

Editar código del backend no requiere reconstruir (está montado como volumen), pero

si tocas `package.json`:

```
docker compose up -d --build backend
```

Sin Docker (opcional)

```
cd backend
npm install
npm start
```

Necesitas Node 20+, un MySQL accesible y las bibliotecas de sistema del Dockerfile.

En Windows compilar `canvas` es engorroso — **con Docker es más fiable**.

Base de datos

MySQL ejecuta automáticamente, en este orden, los scripts montados en

`/docker-entrypoint-initdb.d/`:

Orden	Archivo	Contenido
1	<code>database/init.sql</code>	Estructura principal: roles, usuarios, auditoría
2	<code>database/02-ecommerce.sql</code>	Catálogo, carritos, órdenes, entregas, áreas
3	<code>database/05-seed-users-stock.sql</code>	5 usuarios de prueba y stock variado

Importante: solo se ejecutan cuando el volumen de datos está **vacío**. Si ya levantaste el proyecto antes, no volverán a correr aunque cambies los `.sql`. Para forzar una base limpia: `docker compose down -v`.

Si un script falla, MySQL aborta la cadena y los siguientes no se ejecutan. Por eso el paso 4 comprueba que haya 5 usuarios: es la señal de que los tres corrieron.

Solución de problemas

bind: Intento de acceso a un socket no permitido por sus permisos de acceso

Ocurre en Windows aunque no haya ningún programa usando el puerto. El sistema tiene rangos de puertos reservados (Hyper-V, WSL) y el 8081 suele estar entre ellos.

Compruébalo:

```
netsh interface ipv4 show excludedportrange protocol=tcp
```

Si aparece el 8081, **no cambies** `docker-compose.yml` —es un archivo compartido y el problema es solo de tu máquina—. Crea un `docker-compose.override.yml` en la raíz:

```
services:
  frontend:
    ports: !override
      - "8082:80"
```

Docker Compose lo carga automáticamente. La etiqueta `!override` es necesaria: sin ella, Compose **suma** los puertos en vez de reemplazarlos y el 8081 sigue fallando.

Actualiza también las URLs de tu `.env` al puerto nuevo:

```
PUBLIC_URL=http://localhost:8082
PUBLIC_BASE_URL=http://localhost:8082
FRONTEND_URL=http://localhost:8082
```

Añade `docker-compose.override.yml` a tu `.gitignore` local o simplemente no lo subas: es configuración de tu equipo, no del proyecto.

La base arranca con 2 usuarios en vez de 5

Un script SQL falló y cortó la inicialización:

```
docker compose logs mysql | grep -i error
```

Corrige el script, y después:

```
docker compose down -v && docker compose up -d
```

failed to connect to the docker API

Docker Desktop no está corriendo. Ábrelo y espera a que el icono deje de animarse.

El backend no arranca y los logs mencionan `JWT_SECRET`

Falta `.env` o está incompleto. Revisa el paso 2.

Comandos de uso diario

```
docker compose ps          # estado de los servicios
docker compose logs -f backend # seguir los logs del backend
docker compose restart backend # reiniciar solo el backend
docker compose down        # detener, conservando los datos
docker compose down -v     # detener y BORRAR la base de datos
docker compose up -d --build # reconstruir y levantar
```

Despliegue en producción

Lo que cambia respecto al entorno local:

1. `NODE_ENV=production` y secretos distintos a los de desarrollo.
2. `COOKIE_SECURE=true`, que exige HTTPS.
3. `RECAPTCHA_SKIP=false` con claves reales de reCAPTCHA v2.
4. **Certificado TLS en Nginx.** `frontend/nginx.conf` solo declara `listen 80`:

hay que añadir el bloque 443 con el certificado.

1. **Credenciales de Gmail** (`EMAIL_USER` y `EMAIL_PASS`) para los correos de

verificación y recuperación.

1. **Volumen persistente** para `backend/uploads`, donde se guardan credenciales,

constancias y fotos de entrega.

El workflow `.github/workflows/deploy.yml` incluye un trabajo de despliegue manual

preparado para esto. Antes de habilitarlo hay que definir las variables

`DEPLOY_PATH` y `DEPLOY_DOMAIN` del repositorio y dar de alta el runner. Los

detalles están comentados en el propio archivo.